🤖 AI Job Application Bot - Complete Project Documentation

📋 Project Overview

Project Name: AI-Powered Job Application Automation Bot
Version: 2.0 (Enhanced Multi-Company Edition)
Release Date: December 2025
Technology Stack: Python, Flask, Playwright, Google Gemini AI, Chart.js

---

👨‍💻 Developer Information

Developer: Vignesh Yadala
Email: vignesh.yadala@gmail.com
LinkedIn: linkedin.com/in/vignesh-yadala
GitHub: github.com/Vigneshyadala
Phone: +91 9032478898
Expertise: Python | AI | Automation | Full-Stack Development

🎯 Project Description

An intelligent job application automation system that leverages Google Gemini AI to analyze job postings, match them against candidate resumes, and automate the application process across multiple job portals and company career pages.

Key Features:

- ✅ **AI-Powered Job Matching** using Google Gemini 1.5 Flash
- ✅ **Multi-Platform Support** (Naukri.com + 14 Tech Companies)
- ✅ **Smart Resume Analysis** with PDF parsing
- ✅ **Interactive Web Dashboard** with real-time analytics
- ✅ **Automated Application Submission**
- ✅ **Match Score Analytics** (0-100% compatibility scoring)
- ✅ **Beautiful Data Visualization** using Chart.js
- ✅ **Categorized Company Selection** (Indian IT Giants, FAANG+)

🏗️ System Architecture

Components:

1. Backend (Python Flask)

- **Framework:** Flask 2.0+
- **Web Server:** Development server (port 5000)
- **API Endpoints:** RESTful JSON APIs
- **File Handling:** Resume upload, CSV/JSON storage

2. AI Engine (Google Gemini)

- **Model:** Gemini 1.5 Flash
- **Purpose:** Job-Resume matching analysis
- **Input:** Job description + Resume text
- **Output:** Match score (0-100%), reasoning, recommendation

3. Web Scraping (Playwright)

- **Browser:** Chromium (headless/headful mode)
- **Target Sites:** Naukri.com + 14 company career pages
- **Techniques:** DOM traversal, dynamic content handling, auto-login

4. Frontend (HTML/CSS/JS)

- **Framework:** Vanilla JavaScript
- **Styling:** Custom CSS with gradient themes
- **Charts:** Chart.js for data visualization
- **Design:** Responsive, mobile-friendly interface

5. Data Storage

- **Resume:** PDF files in `data/` folder
- **Results:** CSV files (`job_results_TIMESTAMP.csv`)
- **Logs:** Text files in `logs/` folder
- **Config:** `.env` file for settings

📂 Project Structure

```
ai-job-bot/
│
├── app.py                  # Flask web server
├── enhanced_main.py        # Main bot logic
├── dashboard.py            # CLI dashboard viewer
│
├── templates/
│   └── index.html          # Web interface
│
├── data/
│   ├── resume.pdf          # User resume
│   ├── job_results_*.csv   # Job data
│   └── applied_jobs.json   # Application tracker
│
├── logs/
│   └── bot_YYYYMMDD.log    # Daily logs
│
├── .env                    # Configuration file
├── requirements.txt        # Python dependencies
└── README.md               # Project documentation
```

🔧 Technical Implementation

1. Resume Processing

```python
# Extract text from PDF resume
import PyPDF2

def extract_resume_text(pdf_path):
    with open(pdf_path, 'rb') as file:
        pdf_reader = PyPDF2.PdfReader(file)
        text = ""
        for page in pdf_reader.pages:
            text += page.extract_text()
    return text
```

2. AI-Powered Job Analysis

```python
```

```python
# Google Gemini AI integration
import google.generativeai as genai

genai.configure(api_key=GOOGLE_API_KEY)
model = genai.GenerativeModel('gemini-1.5-flash')

def analyze_job(job_data, resume_text):
    prompt = f"""
    Analyze job match:
    Job: {job_data['title']} at {job_data['company']}
    Resume: {resume_text}

    Provide:
    - Match score (0-100%)
    - Reasoning
    - Apply recommendation
    """
    response = model.generate_content(prompt)
    return parse_ai_response(response.text)
```

3. Web Scraping with Playwright

```python
# Automated job scraping
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    browser = p.chromium.launch(headless=False)
    page = browser.new_page()

    # Navigate and extract jobs
    page.goto("https://www.naukri.com/...")
    job_cards = page.query_selector_all(".job-card")

    for card in job_cards:
        job = {
            'title': card.query_selector('.title').text(),
            'company': card.query_selector('.company').text(),
            'location': card.query_selector('.location').text()
        }
        jobs.append(job)
```

4. Flask API Endpoints

Endpoint	Method	Description
`/`	GET	Main dashboard
`/api/stats`	GET	Get application statistics
`/api/resume`	GET	Get resume info
`/api/resume/upload`	POST	Upload resume PDF
`/api/start-bot`	POST	Start bot with config
`/api/stop-bot`	POST	Stop running bot
`/api/save-settings`	POST	Save bot settings
`/api/load-settings`	GET	Load current settings
`/api/logs`	GET	Get recent logs

🎨 User Interface Features

Dashboard Components:

1. Overview Tab

- **Real-time Statistics:** Total apps, success rate, avg match score
- **Interactive Charts:** Company distribution, score breakdown
- **Recent Activity Feed:** Latest job applications

2. Resume Tab

- **Drag-and-drop Upload:** PDF resume uploader
- **File Validation:** Size and format checking
- **Success Confirmation:** Upload status display

3. Control Panel Tab

- **Company Selection Grid:**
  - 🔵 Job Portal: Naukri.com

    - 🇮🇳 Indian IT Giants: TCS, Infosys, Wipro, HCLTech, Tech Mahindra, Accenture, Cognizant
    - 🌍 FAANG + Big Tech: Google, Microsoft, Amazon, Apple, Meta, Oracle, IBM
- **Smart Selection Features:**
    - Select All by category
    - Clear All selections
    - Visual active state indicators
- **Job Preferences:**
    - Desired roles (comma-separated)
    - Target locations
    - Experience range (min-max years)
- **Start Bot Button:** One-click launch

4. Settings Tab

- **API Configuration:** Google Gemini API key
- **Matching Threshold:** Minimum match score (%)
- **Application Limits:** Max applications per day
- **Save Configuration:** Persist settings to `.env`

🚀 Installation & Setup

Prerequisites:

```bash
Python 3.8+
pip (Python package manager)
Google Gemini API key
```

Step-by-Step Installation:

1. Clone Repository

```bash
git clone https://github.com/Vigneshyadala/ai-job-bot.git
cd ai-job-bot
```

2. Install Dependencies

```bash
pip install -r requirements.txt
playwright install chrome
```

3. Configure Environment

Create `.env` file:

```env
# Google Gemini API
GOOGLE_API_KEY=your_gemini_api_key_here

# Naukri Credentials
NAUKRI_EMAIL=your_email@example.com
NAUKRI_PASSWORD=your_password

# Job Preferences
DESIRED_ROLES=Python Developer,Software Engineer,Data Scientist
DESIRED_LOCATIONS=Bangalore,Mumbai,Chennai,Hyderabad,Pune
MIN_EXPERIENCE=0
MAX_EXPERIENCE=5

# Bot Settings
MIN_MATCH_SCORE=75
MAX_APPLICATIONS_PER_DAY=20
AUTO_APPLY=false
```

4. Prepare Resume

```bash
# Place your resume PDF in data folder
mkdir data
cp your_resume.pdf data/resume.pdf
```

5. Launch Application

```bash
python app.py
```

6. Access Dashboard

Open browser: http://localhost:5000

💻 Usage Guide

Basic Workflow:

1. Upload Resume

- Navigate to "Resume" tab
- Click upload area or drag PDF
- Confirm successful upload

2. Configure Settings

- Go to "Settings" tab
- Enter Google Gemini API key
- Set match score threshold (recommended: 75%)
- Set max applications (recommended: 20/day)
- Click "Save Settings"

3. Select Companies

- Open "Control Panel" tab
- Choose companies individually or use "Select All" for categories
- Enter desired job roles (e.g., "Python Developer, Software Engineer")
- Enter target locations (e.g., "Bangalore, Mumbai")
- Set experience range

4. Start Bot

- Review selections (check counter shows correct number)
- Click "Start Job Application Bot"
- Bot opens browser and begins scraping
- Monitor progress in terminal/logs

5. View Results

- Switch to "Overview" tab

- View statistics and charts
- Refresh data periodically
- Check detailed results in CSV files

📊 Data Analytics

Metrics Tracked:

Metric	Description
Total Applications	Count of all jobs analyzed
Successful Applications	Jobs where bot applied
Average Match Score	Mean compatibility percentage
Company Distribution	Jobs per company breakdown
Score Distribution	Jobs by score ranges (90-100%, 75-89%, etc.)
Location Analysis	Jobs by geographic location
Timeline	Applications over time

Export Formats:

- **CSV:** `job_results_YYYYMMDD_HHMMSS.csv`
- **JSON:** `job_results_YYYYMMDD_HHMMSS.json`

CSV Columns:

```
title, company, location, experience, url, job_id, source,
search_role, search_location, description, match_score,
reasoning, should_apply, applied, timestamp
```

🔐 Security & Privacy

Data Handling:

- ✅ **Local Storage:** All data stored locally on user machine

- ✅ **No Cloud Sync:** Resume and personal data never uploaded to cloud
- ✅ **API Key Protection:** Keys stored in `.env` (git-ignored)
- ✅ **Secure HTTPS:** API calls use HTTPS encryption
- ✅ **Password Handling:** Naukri password stored locally only

Best Practices:

1. Never commit `.env` file to version control
2. Use strong, unique API keys
3. Regularly rotate passwords
4. Review logs for suspicious activity
5. Keep dependencies updated

🐛 Troubleshooting

Common Issues:

1. Bot fails to start

Solution: Check API key in `.env`, verify internet connection

2. Resume upload fails

Solution: Ensure PDF is under 16MB, contains searchable text

3. No jobs found

Solution: Try different roles/locations, check spelling

4. Login fails on Naukri

Solution: Verify credentials, complete manual CAPTCHA if needed

5. Charts not displaying

Solution: Clear browser cache, check browser console for errors

6. Playwright browser issues

Solution: Run `playwright install chrome` again

🔄 Future Enhancements

Planned Features:

- ☐ **LinkedIn Integration:** Apply directly on LinkedIn
- ☐ **Indeed Support:** Add Indeed.com scraping
- ☐ **Email Notifications:** Send daily summary reports
- ☐ **Multi-Resume Support:** Handle different resumes for different roles
- ☐ **Interview Scheduler:** Automatically schedule interviews
- ☐ **Cover Letter Generator:** AI-generated custom cover letters
- ☐ **Application Tracking:** Follow-up reminders and status updates
- ☐ **Mobile App:** React Native mobile version
- ☐ **Docker Support:** Containerized deployment
- ☐ **Cloud Deployment:** Heroku/AWS deployment guide

📈 Performance Metrics

Benchmarks:

- **Scraping Speed:** ~10-15 jobs per minute
- **AI Analysis:** ~2-3 seconds per job
- **Resume Processing:** <1 second per PDF
- **Dashboard Load Time:** <2 seconds
- **Memory Usage:** ~200-300 MB (browser included)
- **CPU Usage:** 10-20% on modern systems

Scalability:

- Can handle 1000+ jobs per session
- Supports multiple concurrent browser tabs
- Efficient data caching and storage

📚 Dependencies

Python Packages:

```
Flask==2.3.0
```

```
pandas==2.0.0
python-dotenv==1.0.0
playwright==1.40.0
google-generativeai==0.3.0
PyPDF2==3.0.1
werkzeug==2.3.0
```

Frontend Libraries:

```
Chart.js==4.4.0 (CDN)
```

System Requirements:

- **OS:** Windows 10/11, macOS 10.15+, Linux (Ubuntu 20.04+)
- **RAM:** 4GB minimum, 8GB recommended
- **Storage:** 500MB for installation + data
- **Browser:** Chrome/Chromium (auto-installed by Playwright)

📜 License & Legal

License:

MIT License - Free for personal and commercial use

Terms of Use:

1. Use responsibly and ethically
2. Respect website terms of service
3. Don't spam or abuse job portals
4. Provide accurate information in applications
5. Follow local employment laws

Disclaimer:

This tool is for **educational and personal use only**. The developer is not responsible for:

- Misuse of the application
- Violations of website terms of service
- Job application outcomes

- Data accuracy or completeness

🤝 Contributing

How to Contribute:

1. Fork the repository
2. Create feature branch (`git checkout -b feature/NewFeature`)
3. Commit changes (`git commit -m 'Add NewFeature'`)
4. Push to branch (`git push origin feature/NewFeature`)
5. Open Pull Request

Contribution Guidelines:

- Follow PEP 8 Python style guide
- Add comments for complex logic
- Update documentation for new features
- Test thoroughly before submitting
- Include examples in PR description

📞 Support & Contact

Get Help:

- **Email:** vignesh.yadala@gmail.com
- **LinkedIn:** linkedin.com/in/vignesh-yadala
- **GitHub Issues:** github.com/Vigneshyadala/ai-job-bot/issues
- **Phone:** +91 9032478898

Response Time:

- Email: Within 24-48 hours
- GitHub Issues: Within 2-3 days
- LinkedIn: Within 1-2 days

🌟 Acknowledgments

Technologies Used:

- **Google Gemini AI** - Powerful LLM for job matching
- **Playwright** - Modern web automation framework
- **Flask** - Lightweight Python web framework
- **Chart.js** - Beautiful data visualization library

Inspiration:

Built to help job seekers automate the tedious application process and focus on interview preparation.

📊 Project Statistics

Metric	Value
Lines of Code	~2,500
Files	8 core files
API Endpoints	10 endpoints
Supported Companies	15 platforms
Development Time	40+ hours
Version	2.0
Last Updated	December 2025

🎓 Learning Outcomes

Skills Demonstrated:

1. **Web Scraping** - Advanced DOM manipulation
2. **AI Integration** - LLM API usage
3. **Full-Stack Development** - Backend + Frontend
4. **Data Analysis** - CSV processing, statistics

5. **UI/UX Design** - Responsive, modern interface
6. **API Development** - RESTful architecture
7. **Automation** - Browser automation, task scheduling
8. **File Handling** - PDF parsing, data storage

💡 Tips for Success

Maximize Results:

1. **Optimize Resume:** Use keywords from job descriptions
2. **Target Smartly:** Focus on relevant roles and locations
3. **Set Realistic Thresholds:** 75% match score is ideal
4. **Time Your Applications:** Apply during business hours
5. **Follow Up:** Check application status regularly
6. **Customize Approach:** Adjust settings based on results
7. **Stay Updated:** Keep resume current with new skills
8. **Be Patient:** Quality over quantity in applications

🚨 Important Notes

Ethical Usage:

- ⚠️ Always provide accurate information
- ⚠️ Don't spam companies with multiple applications
- ⚠️ Respect rate limits on job portals
- ⚠️ Use for genuine job search only
- ⚠️ Review applications before submission

Rate Limiting:

- Naukri.com: ~100 searches per day recommended
- Company sites: 2-3 second delays between requests
- API calls: Google Gemini has generous free tier

📅 Version History

Version 2.0 (December 2025) - Current

- ✨ Enhanced multi-company support (15 platforms)
- ✨ Categorized company selection UI
- ✨ Improved AI matching algorithm
- ✨ Real-time dashboard analytics
- ✨ Better error handling and logging
- ✨ Responsive mobile design

Version 1.0 (November 2025)

- 🎉 Initial release
- ✅ Basic Naukri.com scraping
- ✅ Simple AI matching
- ✅ Command-line interface

🎯 Success Stories

> *"Got 3 interview calls in the first week of using this bot!"*
> — Anonymous User, Software Engineer

> *"Saved hours of manual applications. The AI matching is surprisingly accurate."*
> — Anonymous User, Data Scientist

📎 Related Projects

Similar Tools:

- **LinkedIn Easy Apply Bots** - LinkedIn automation
- **Resume Parser Tools** - Extract resume data
- **Job Board Aggregators** - Collect jobs from multiple sites

Complementary Tools:

- **Grammarly** - Improve application quality

- **Calendly** - Schedule interviews easily
- **Notion** - Track application status

🏆 Awards & Recognition

- 🥇 Featured in college tech fest 2025
- 🎖️ Used by 50+ job seekers successfully
- ⭐ GitHub stars: Growing community

📖 Additional Resources

Learning Materials:

- Python Web Scraping Guide
- Flask Documentation
- Playwright Docs
- Google AI Studio

Job Search Tips:

- Resume Writing Guide
- Interview Preparation
- Salary Negotiation

🎉 Final Words

Thank you for using the **AI Job Application Bot**! This project represents the intersection of AI, automation, and practical problem-solving.

Whether you're a job seeker looking to streamline your application process or a developer interested in learning about web automation and AI integration, I hope this tool serves you well.

Remember: This tool is meant to assist, not replace, your job search efforts. Always review applications, customize your approach, and maintain authenticity in your interactions with potential employers.

📧 Stay Connected

Follow for updates, new features, and more automation projects:

Vignesh Yadala
📧 vignesh.yadala@gmail.com
💼 LinkedIn
📎 GitHub
📱 +91 9032478898

Last Updated: December 14, 2025
Documentation Version: 2.0
© 2025 Vignesh Yadala. All rights reserved.

Made with ❤️ and Python by Vignesh Yadala